

ASSIGNMENT-20.3

2303A52140

BATCH-41

NAME:Ch.Rohith reddy

Task 1 – Password Strength & Validation Check

PROMPT: Password Strength & Validation Check

Requirements:

1. Create a function to check the strength of a password based on the following criteria

- Minimum length of 8 characters.

- Contains at least one uppercase letter.

- Contains at least one lowercase letter.

- Contains at least one digit.

2. The function should return a message indicating whether the password is "Strong", "Moderate", or "Weak".

3. Handle edge cases (e.g., empty string, only special characters) appropriately

```
E: > ai ac.py > ...
1  #Password Strength & Validation Check
2  # Requirements:
3  # 1. Create a function to check the strength of a password based on the following criteria
4  #   - Minimum length of 8 characters.
5  #   - Contains at least one uppercase letter.
6  #   - Contains at least one lowercase letter.
7  #   - Contains at least one digit.
8  # 2. The function should return a message indicating whether the password is "Strong", "Moderate", or "Weak".
9  # 3. Handle edge cases (e.g., empty string, only special characters) appropriately
10 def check_password_strength(password):
11     """
12     Check the strength of a given password based on specific criteria.
13     Parameters:
14     password (str): The password string to be evaluated.
15     Returns:
16     str: A message indicating whether the password is "Strong", "Moderate", or "Weak".
17     Criteria:
18     - Strong: Meets all criteria (length >= 8, contains uppercase, lowercase, and digit).
19     - Moderate: Meets at least two of the criteria.
20     - Weak: Meets fewer than two criteria.
21     """
22     # Initialize criteria counters
23     has_uppercase = any(char.isupper() for char in password)
24     has_lowercase = any(char.islower() for char in password)
25     has_digit = any(char.isdigit() for char in password)
26     # Check the length of the password
27     is_long_enough = len(password) >= 8
```

```

21     """
22     # Initialize criteria counters
23     has_uppercase = any(char.isupper() for char in password)
24     has_lowercase = any(char.islower() for char in password)
25     has_digit = any(char.isdigit() for char in password)
26     # Check the length of the password
27     is_long_enough = len(password) >= 8
28     # Count how many criteria are met
29     criteria_met = sum([has_uppercase, has_lowercase, has_digit, is_long_enough])
30     # Determine password strength based on criteria met
31     if criteria_met == 4:
32         return "Strong"
33     elif criteria_met >= 2:
34         return "Moderate"
35     else:
36         return "Weak"
37 # Example usage:
38 if __name__ == "__main__":
39     test_password = "yashwanth@2486"
40     print(f"The password '{test_password}' is: {check_password_strength(test_password)}")
41

```

OUTPUT:

```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\
y"
The password 'Password@2486' is: Strong
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> █

```

Task 2 – Insecure File Upload Handling

PROMPT: # Requirements:

- # 1. Create a function to handle file uploads securely.
- # 2. Validate the file type (e.g., only allow .jpg, .png, .pdf).
- # 3. Check the file size (e.g., limit to 5MB).
- # 4. Sanitize the file name to prevent directory traversal attacks.
- # 5. Store the file in a designated directory with a unique name to avoid overwriting

```

42 #Insecure File Upload handling
43 # Requirements:
44 # 1. Create a function to handle file uploads securely.
45 # 2. Validate the file type (e.g., only allow .jpg, .png, .pdf).
46 # 3. Check the file size (e.g., limit to 5MB).
47 # 4. Sanitize the file name to prevent directory traversal attacks.
48 # 5. Store the file in a designated directory with a unique name to avoid overwriting
49 import os
50 import uuid
51 def secure_file_upload(file_name, file_content):
52     """
53     Handle file uploads securely by validating file type, size, and sanitizing the file name.
54     Parameters:
55     file_name (str): The original name of the uploaded file.
56     file_content (bytes): The content of the uploaded file.
57     Returns:
58     str: A message indicating the result of the upload process.
59     Criteria:
60     - Allowed file types: .jpg, .png, .pdf
61     - Maximum file size: 5MB
62     - Sanitized file name to prevent directory traversal attacks.
63     - Unique file name to avoid overwriting existing files.
64     """
65     # Define allowed file extensions and maximum file size
66     allowed_extensions = {'.jpg', '.png', '.pdf'}
67     max_file_size = 5 * 1024 * 1024 # 5MB in bytes
68     # Extract the file extension
69     _, file_extension = os.path.splitext(file_name)
70     # Validate the file type
71     if file_extension.lower() not in allowed_extensions:
72         return "Invalid file type. Only .jpg, .png, and .pdf are allowed."
73
74     if len(file_content) > max_file_size:
75         return "File size exceeds the limit of 5MB."
76     # Sanitize the file name to prevent directory traversal attacks
77     sanitized_file_name = os.path.basename(file_name)
78     # Generate a unique file name to avoid overwriting existing files
79     unique_file_name = f"{uuid.uuid4()}{file_extension}"
80     # Define the directory to store uploaded files
81     upload_directory = "uploads"
82     # Ensure the upload directory exists
83     os.makedirs(upload_directory, exist_ok=True)
84     # Save the file securely
85     with open(os.path.join(upload_directory, unique_file_name), 'wb') as f:
86         f.write(file_content)
87     return f"File uploaded successfully as {unique_file_name}."
88 # Example usage:
89 if __name__ == "__main__":
90     # Simulate file upload
91     test_file_name = "example.pdf"
92     test_file_content = b"%PDF-1.4 example content" # Simulated file content in bytes
93     print(secure_file_upload(test_file_name, test_file_content))

```

OUTPUT:

```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.p
y"
File uploaded successfully as 8f6d3cc4-34ce-460c-a721-87a0e9f82f01.pdf.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> █

```

Task 3 – Real-Time Application: API Security Review

Requirements:

1. Create a function to review API security based on common vulnerabilities.

2. Check for vulnerabilities such as:

- SQL Injection

- Cross-Site Scripting (XSS)

- Insecure Direct Object References (IDOR)

3. The function should return a report indicating the presence of any vulnerabilities and recommendations for mitigation

```

94
95 #Real-Time Application: API Security review
96 # Requirements:
97 # 1. Create a function to review API security based on common vulnerabilities.
98 # 2. Check for vulnerabilities such as:
99 #    - SQL Injection
100 #    - Cross-Site Scripting (XSS)
101 #    - Insecure Direct Object References (IDOR)
102 # 3. The function should return a report indicating the presence of any vulnerabilities and recommendations for mitigation.
103 def review_api_security(api_endpoint):
104     """
105     Review API security based on common vulnerabilities such as SQL Injection, XSS, and IDOR.
106     Parameters:
107     api_endpoint (str): The API endpoint to be reviewed for security vulnerabilities.
108     Returns:
109     str: A report indicating the presence of any vulnerabilities and recommendations for mitigation.
110     """
111     # Simulated vulnerability checks (In a real implementation, this would involve more complex logic)
112     vulnerabilities = []
113     # Check for SQL Injection vulnerability
114     if "SELECT" in api_endpoint.upper() or "DROP" in api_endpoint.upper():
115         vulnerabilities.append("SQL Injection detected. Recommendation: Use parameterized queries.")
116     # Check for Cross-Site Scripting (XSS) vulnerability
117     if "<script>" in api_endpoint.lower():
118         vulnerabilities.append("XSS detected. Recommendation: Sanitize user input and use Content Security Policy (CSP).")
119     # Check for Insecure Direct Object References (IDOR) vulnerability
120     if "/user/" in api_endpoint.lower() and any(char.isdigit() for char in api_endpoint):
121         vulnerabilities.append("IDOR detected. Recommendation: Implement proper access controls and authorization checks.")
122     # Generate the report
123     if vulnerabilities:
124         report = "Vulnerabilities found:\n" + "\n".join(vulnerabilities)
125     else:
126         report = "No vulnerabilities detected. The API endpoint appears to be secure."
127     return report

```

```

# Example usage:
if __name__ == "__main__":
    test_api_endpoint = "/user/123?name=<script>alert('XSS')</script>"
    print(review_api_security(test_api_endpoint))

```

OUTPUT:

```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai/ac.py"
Vulnerabilities found:
XSS detected. Recommendation: Sanitize user input and use Content Security Policy (CSP).
IDOR detected. Recommendation: Implement proper access controls and authorization checks.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>

```

Task 4 – Cross-Site Scripting (XSS) Check

PROMPT: Requirements:

- # 1. Create a function to check for potential XSS vulnerabilities in user input.
- # 2. The function should analyze the input for common XSS attack vectors (e.g., <script> tags, event handlers).
- # 3. The function should return a message indicating whether the input is safe or potentially vulnerable to XSS attacks


```

131 |     print(review_api_security(test_api_endpoint))'''
132 | #Cross-Site Scripting (XSS) Check
133 | # Requirements:
134 | # 1. Create a function to check for potential XSS vulnerabilities in user input.
135 | # 2. The function should analyze the input for common XSS attack vectors (e.g., <script> tags, event handlers).
136 | # 3. The function should return a message indicating whether the input is safe or potentially vulnerable to XSS attacks.
137 | def check_xss_vulnerability(user_input):
138 |     """
139 |     Check for potential Cross-Site Scripting (XSS) vulnerabilities in user input.
140 |     Parameters:
141 |     user_input (str): The input string to be analyzed for XSS vulnerabilities.
142 |     Returns:
143 |     str: A message indicating whether the input is safe or potentially vulnerable to XSS attacks.
144 |     """
145 |     # Define common XSS attack vectors
146 |     xss_attack_vectors = ["<script>", "onerror=", "onload=", "javascript:"]
147 |     # Check if any of the attack vectors are present in the user input
148 |     for vector in xss_attack_vectors:
149 |         if vector in user_input.lower():
150 |             return "Potential XSS vulnerability detected. Input is potentially unsafe."
151 |     return "Input is safe from XSS vulnerabilities."
152 | # Example usage:
153 | if __name__ == "__main__":
154 |     test_input = "<script>alert('XSS')</script>"
155 |     print(check_xss_vulnerability(test_input))
156 |

```

OUTPUT:



```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.py"
Potential XSS vulnerability detected. Input is potentially unsafe.
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>

```

Task 5 – SQL Injection Prevention

PROMPT: Requirements:

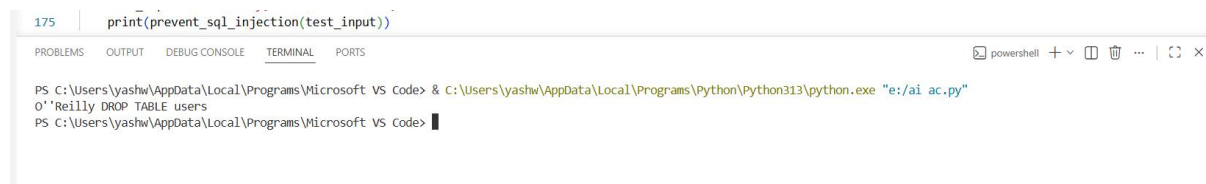
- # 1. Create a function to prevent SQL injection attacks by sanitizing user input.
- # 2. The function should escape special characters and use parameterized queries

```

156 |
157 | # SQL Injection Prevention
158 | # Requirements:
159 | # 1. Create a function to prevent SQL injection attacks by sanitizing user input.
160 | # 2. The function should escape special characters and use parameterized queries.
161 | def prevent_sql_injection(user_input):
162 |     """
163 |     Prevent SQL injection attacks by sanitizing user input.
164 |     Parameters:
165 |     user_input (str): The input string to be sanitized for SQL injection prevention.
166 |     Returns:
167 |     str: A sanitized version of the input string that is safe for use in SQL queries.
168 |     """
169 |     # Escape special characters commonly used in SQL injection attacks
170 |     sanitized_input = user_input.replace("'", "'").replace(";", ";").replace("--", "")
171 |     return sanitized_input
172 | # Example usage:
173 | if __name__ == "__main__":
174 |     test_input = "O'Reilly; DROP TABLE users; --"
175 |     print(prevent_sql_injection(test_input))

```

OUTPUT:



```

PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yashw\AppData\Local\Programs\Python\Python313\python.exe "e:/ai ac.py"
O'Reilly DROP TABLE users
PS C:\Users\yashw\AppData\Local\Programs\Microsoft VS Code>

```

